

Vetting Dry Run

One applicant, one vetter, one statement. What to set up on the community, what to press on each side, and what must be true at every stop along the way.

Design docs/design/vetting-process.md §4 vtc-service 0.11.58 vta-service 0.27.0
vta-sdk 0.38 · vetting feature Actors admin · alice (applicant) · bob (vetter)

Before you start

Vetting is a three-party flow, so most of what goes wrong is topology, not protocol. Settle these first.

- **Bob is already a member** of the community and Alice is not. Only a current member can be named a vetter, and only a non-member has a join to be vetted for.
- **Two VTAs, not one.** Each openvtc profile authenticates to its VTA with its own credential bundle, and `device/register` refuses a re-claim by spec. Run `openvtc -p alice` and `openvtc -p bob` — a profile is a config file plus an OS credential keyed on the profile name, so two on one machine is fine — but point them at separate VTAs.
- **Put both personas on the same mediator** for the first run. Cross-mediator relay of an anon-crypt reply has a known refusal (`session_mismatch`); test that deliberately as a variant, not accidentally as your first run.
- **Alice needs a face carrying `name.legal` with a readable value.** A predicate-only release is refused — a vetter has to read the value off a document.
- Preflight both sides with `openvtc -p alice health` and `openvtc -p bob health`: it resolves the whole messaging chain and says which transport each pair negotiates.
- **Prove the community will admit before you involve two people** — see below.

Dry-run the verdict first

ADMIN

In the admin console: **Ceremonies → Join → Flow**. Set *Peer identity vetting* to *requirements met* and read where the path lands.

EXPECT

`ADMIT · as member`. The chart is drawn by evaluating the community's live policy, so this is the same verdict Alice's submission will get.

If it lands on `REFER · to the moderator queue` instead, the join policy has no vetting route — an older community keeps the policy installed at its first boot, and the shipped defaults are only installed for a purpose that has none. Nothing later in this document will work until that is fixed: take `vtc-service/policies/default/join.rego`, paste it into *Upload raw Rego*, and **activate** the new revision (uploading does not activate it).

Worth a second look while you are there: set vetting to *not required* and watch the invitation and trusted-credential questions become the live path. That is the bypass described under step 02 — visible rather than inferred.

One vetter is the shortest useful run, not the most revealing. With `minStatements: 1`, independence and commitment-consistency checks are vacuous. Do the one-vetter run first to prove the wire works, then raise it to 2 (see the negative cases) to exercise the counting.

Phase 1 — Tell the community what it requires

All of this is the admin console's **Vetting → Requirements** page. The requests shown are what it sends, and remain the way to script a community's setup. Nothing about vetting is on by default; every number below is your policy.

01

Register the statement type

ADMIN

Vetting Statements are `EndorsementCredential`s of a registered endorsement type. A criterion cannot name an unregistered type.

The Requirements page notices when the identity-vetting type is missing and offers **Register it** — one button, because this is the step whose absence surfaces later as a refusal of something else. Or:

```
POST /v1/endorsement-types
( Trust Task https://trusttasks.org/spec/vtc/endorsement-type

{
  "typeUri": "https://firstperson.network/endorsements/identity",
  "description": "A member verified this person's identity"
}
```

02

Publish a criterion that needs one vetter

ADMIN

This is the answer to “how do I change the join ceremony to allow one vouching member”: the count lives on an `Accepts` criterion, not in the ceremony or in Rego. `join.rego` already reads the vetting facts and needs no edit.

Add a criterion opens a form over every field below — a new one starts at one vetter, in person or on video, verifying a legal name, which is this run's policy already. It checks what you type against the same rules the daemon applies and will not send requirements the community would refuse. The equivalent request:

```
POST /v1/schemas/accepts

{
  "id": "vetted-member",
  "description": "One vetter must confirm who you are",
  "query": { "credentials": [ { "id": "vetting", "format": "local",
    "meta": { "type_values": ["EndorsementCredential"]
  "vetting": {
```

```

    "version": "0.1",
    "statementType": "https://firstperson.network/endorsements",
    "minStatements": 1,
    "acceptedMethods": ["inPerson", "video"],
    "requiredClaims": ["name.legal"],
    "maxStatementAge": "P120D",
    "eligibleVetters": { "role": "vetter" },
    "independence": { "requireConsistentIdentityCommitment": true }
  }
}

```

- The route refuses requirements nobody could satisfy: `minStatements: 0`, a `minByMethod` floor on a method missing from `acceptedMethods`, a month-based duration (write `P120D`, not `P4M`), and an unregistered `statementType`.
- Leave `invitation` out — only `"required"` makes the applicant also present a VIC.
- **Delete or disable any other criterion that would admit Alice without vetting**, or she may be admitted down the easier path and the run proves nothing.

03

ADMIN

Confirm the applicant can read it

The saved criterion appears on the same page with its requirements in sentences and its `requirementsDigest` beside them — the digest comes from `vtc/join-requests/manifest/0.2`, which is what Alice actually reads, so what you see there is what she gets. Over REST that is `GET /v1/schemas/accepts` for the record and the manifest task for the digest.

EXPECT

the manifest criterion carries a `vetting` object and a `requirementsDigest`; `manifest/0.1` is unchanged. Read the summary sentences back before going further — they are the thing Alice will be held to.

Then return to **Ceremonies → Join → Flow** once more. With the criterion saved, the chart is drawn against a community that actually vets: the vetting questions should now be the ones deciding the outcome. A chart that still turns only on invitations and credentials means this criterion is not the one an applicant meets.

These console pages are new. Until VTI #1465 and #1467 landed on 14 September 2026 the Requirements page was read-only and the two calls above had no interface at all. They are covered by component tests but have not been exercised against a live daemon, so if a page disagrees with you, the REST requests above are the same thing and the daemon is the authority either way.

Phase 2 — Name Bob an approved vetter

A vetter is a member the community has named one. The grant is a revocable `CommunityRole: vetter` credential the community issues and delivers to the member.

04

ADMIN

Grant the role

Three equivalent surfaces: the member's page in the admin console (“Grant vetter role”), the CLI, or the Trust Task over REST / DIDComm / TSP.

```
cnm vetting vetters grant did:webvh:...:bob --validity 180d

# or: POST /v1/vetting/vetters
#      ( vtc/vetting/vetters/grant/0.1 )
#      { "memberDid": "did:webvh:...:bob", "validitySeconds": 155
```

Validity defaults to one year; the accepted range is one day to two years. Granting again while a live grant exists returns that grant rather than issuing a second.

EXPECT

`cnm vetting vetters list` shows Bob with `live: true`, `origin: "manual"`, and an audit entry `VetterGranted` with a `VecIssued` beside it.

05

BOB

Check the credential landed in his wallet

The community delivers the role credential over `credential-exchange/issue`. If Bob's client was offline it will not arrive on its own.

Vetting Tickets tab `g` ask the community to send it again

The admin equivalent is `cnm vetting vetters resend <memberDid>` or `POST /v1/vetting/vetters/{memberDid}/resend`. Nothing new is issued either way — it is the same credential.

Variant worth a second run: automatic grants. The `vetterEligibility` policy can be authored as ordered rules under **Ceremonies** — active, under review, how they were admitted, tenure, vetting depth, roles held — or uploaded as Rego into package `vtc.vetter_eligibility`. Then turn the sweep on, in **Vetting** → **Automatic grants** or with `cnm vetting auto-grant set --enabled true --sweep-minutes 60`. The shipped default names only genesis members, so a policy of your own is the point of the exercise.

Phase 3 — The run

Both people at their terminals, ideally on a call, because the session has a spoken step. Keys below are the Vetting page's; each tab shows its own verbs in the footer.

06

BOB

Open the desk and cut a ticket

A ticket is the vetter's consent to be asked; without one the request

never becomes a task. Defaults are one use and fourteen days.

t new ticket **u** copy the vetting-ticket: link
y copy the XXXX-XXXX code **p** publish a vetter profile

The selected ticket is drawn as a QR of its link. Publishing a listed profile is optional here, but it is what lets Alice find Bob with **v** instead of being handed a link.

07

Start the application and ask

ALICE

n start application **f** choose face
m refresh requirements **v** find vetters
r request a vetter

Starting the application is where the join DID is chosen or minted — before gathering, so every card, statement and the final presentation name the same DID. Pasting Bob's ticket link into the request form fills in his DID and the ticket together; a link for another community is refused before anything is sent.

EXPECT

Bob's Desk tab shows the request already **Accepted** — acceptance is automatic, the ticket was the consent — and Alice's row reads *named a vetter until ... with not revoked when checked on ...* once the background status check returns.

08

Open the session — and read the code aloud

BOB

o open session **x** decline

Both screens now show the same XXXX-XXXX match code, derived from the session document id under the tag `vetting-session-match/v1`. Reading it to each other is what ties the person on the call to the client holding the join DID. It is domain-separated from the personhood ceremony, so the same DID must *not* produce the same

personhood ceremony, so the same DID must not produce the same code in both — worth checking once.

09

Review and send the card

ALICE

c or **Enter** review card

The disclosure preview shows exactly what Bob will see. Nothing leaves until Alice confirms; a `stepUpRequired` refusal leaves the preview intact so she can approve on her device and present the same preview again. The card is then signed as the join persona DID.

EXPECT

Bob's desk moves to `CardReceived`, and the card verifies against the session's challenge, audience and community.

10

Check the human, then attest

BOB

a or **Enter** check and attest

Mark `name.legal` verified against whatever document Bob accepts, record which documentation he relied on, and confirm liveness. Signing is never automatic — it is attributable to his member DID.

Deliberately trip each refusal once: attesting before confirming the match code, marking a claim the card does not carry, leaving a required claim unverified, and leaving documentation blank. Each has its own message.

EXPECT

a signed statement issued to Alice, which her client verifies against her own card before storing it.

11

Submit and be admitted

ALICE

The checklist reads 1 of 1 and says *meets the published requirements*

— advisory wording, because only the community knows whether Bob was eligible. Submitting rides the normal join: the statement goes in the presentation and the requirementsDigest goes in extensions .

EXPECT

verdict allow → VMC grant + role VEC → reciprocal VMC. On the admin side, the decision records the vetting facts it relied on and which requirements version governed it.

What to assert, and where

STOP	CLIENT SIDE	COMMUNITY SIDE
Requirements read	Alice's vetting page names one statement and name.legal ; digest stored	manifest/0.2 carries vetting + requirementsDigest
Vetter named	Bob holds a CommunityRole: vetter VEC with a credentialStatus	cnm vetting vetters list → live: true , origin: manual
Request accepted	Eligibility presentation verifies; grant recorded as not revoked	—
Session open	Same match code on both screens; differs from a personhood code for the same id	—
Statement issued	Statement verifies against Alice's card; checklist 1/1	—
Verdict	Membership appears; reciprocal VMC issued	Decision records vetting facts + requirements version

Negative cases

Ordered by what they actually catch. The first four are cheap; the last two are where the interesting defects live.

ANTI-SPAM

Ask with no ticket

The gate runs before a task is created, so the request should never reach Bob's desk at all. Five bad codes from one sender in an hour drops that sender silently for the hour.

ANTI-SPAM

Ticket from another community

Alice's client refuses it before sending — sending would tie her join DID to a community she is not applying to.

COUNTING

Raise to two statements

Submit with one. The verdict should be `request_more` with the generic `vetting` need expanded to `vetting:statements:1`. Gather a second and submit again.

COUNTING

Statement past `maxStatementAge`

It stops counting at submit without invalidating anything else.

REVOCATION

Revoke the grant before submit

`cnm vetting vetters revoke <endorsementId>`. Alice's row should flip to *the community has revoked this vetter's grant*, and the community should stop counting the statement. Bob's profile is deleted with the grant.

REVOCATION

Withdraw after admission

Bob presses w on the Issued tab. `cnm vetting revocations` should show Alice's admission as `needsReview` — flagged, not removed.

CONSISTENCY

Two vetters, two identities

With two statements, change the `name.legal` value between cards. The commitments diverge and the verdict should be `refer` to the `vetting-review` queue, not a denial.

NETWORKING

Stop the VTC mid-question

A manifest, directory or resend question unanswered after 30 s must be *reported as unanswered*, never hang (R1.2) — and the message must tell unreachable from auth-rejected from contract-mismatch (R6.4).

TOPOLOGY

Put Bob on a second mediator

Run the whole flow across mediators. If a reply vanishes, go to the *sending* mediator's `FORWARD_` lines — a state of `Sent` is not delivery.

What is already covered, and the gap

- **Server side, end to end:** `vtc-service/tests/vetting_journey.rs` runs the whole story in process — admin setup, a manual grant and an automatic sweep, vetter profiles and directory filters, two statements, submit, withdrawal, and a revoked grant. Read it as the executable version of this document.
- **Client side, in pieces:** roughly forty unit tests in `openvtc-core/src/vetting/tests.rs` cover ticket redemption, requirement evaluation, card and statement verification, and the community-query matching.
- **The gap:** `openvtc-core/tests/` has no vetting end-to-end. Every other flow of this size has one — `join_lifecycle_e2e.rs`, `community_context_e2e.rs`, `relationship_e2e.rs`. Until a MockVta-backed `vetting_e2e.rs` mirrors phase 3, the manual run above is the only client-side proof, and it is not repeatable. Note that those e2es run only under `--include-ignored`, which CI does on the Coverage job alone.

Steps and keys read from `openvtc-core/src/vetting/` and the Vetting page's key handler; admin surfaces from `docs/03-vtc/vetting.md`, the admin console and `cnm-cli` in the verifiable-trust-infrastructure tree. Requirement semantics follow `docs/design/vetting-process.md` §6 and §10. Revised 14 September 2026: VTI #1465 and #1467 moved community setup into the console, #1471 gave the join dry-run its vetting inputs, and #1472 added the Flow view that step 00 relies on.

